ColumnCast: Broadcasting columns to places of interest!

Pratyoy Das
pratyoyd@uci.edu
University of California
Irvine, California, USA

Vishal Chakraborty
vchakrab@uci.edu
University of California
Irvine, California, USA

Sharad Mehrotra
sharad@ics.uci.edu
University of California
Irvine, California, USA

Abstract

Joins between large fact tables and dimension tables remain a major cost in analytical workloads. A common workaround is to replace normalized schemas with fully denormalized one-big-table layouts, but this sacrifices logical independence and incurs substantial storage and maintenance overhead. We argue that full denormalization is often unnecessarily coarse: in many recurring workloads, only a small subset of dimension attributes repeatedly matters. We present ColumnCast, a workload-driven, storage-budgeted framework that selectively grafts such dimension attributes into fact tables while preserving the normalized logical schema. Grafted attributes can either eliminate joins entirely or enable predicates to be evaluated directly on the fact table, reducing I/O and shrinking the data that reaches remaining joins. ColumnCast operates in two stages: an offline optimizer selects a budget-feasible set of grafts from workload logs, and an online graft-aware rewriter uses those grafts only when they improve execution. On the Star Schema Benchmark, ColumnCast delivers an overall 1.41× speedup with only 8% storage overhead, showing that a small number of carefully chosen grafts can recover much of the performance benefit of denormalization without its full cost. This positions selective grafting as a practical middle ground between normalized schemas and one-big-table designs, while opening up many research opportunities in workload-aware physical design, adaptive maintenance, optimizer integration, and new storage layouts for modern data systems.

ACM Reference Format:
Pratyoy Das, Vishal Chakraborty, and Sharad Mehrotra. 2018. ColumnCast: Broadcasting columns to places of interest!. In . ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The decoupling of compute and storage in modern analytical engines has accelerated the adoption of open table formats such as Apache Iceberg [], Delta Lake [], and Apache Hudi []. Yet an old bottleneck remains: queries over normalized schemas still pay substantial cost to repeatedly join large fact tables with dimensions, especially in read-heavy analytical settings. [].

In practice, this cost is often avoided through fully denormalized one-big-table (OBT) layouts []. OBTs replace joins with local scans, but they do so by denormalizing *everything*. This comes with drawbacks: wider scans increase decompression and tuple-materialization costs, updates to dimension attributes amplify writes into fact storage, and the benefits of normalized design are lost. []. More importantly, full denormalization is often unnecessarily coarse, since many workloads repeatedly touch only a small subset of dimension attributes.

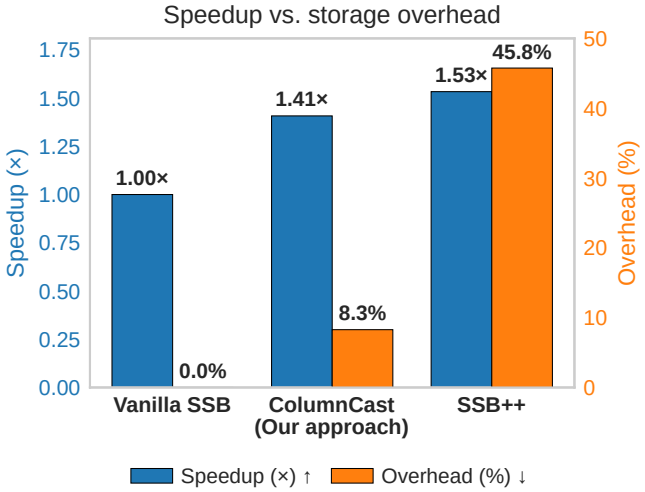

Figure 1: Performance of ColumnCast against baselines.

This paper explores a middle ground. We observe that analytical workloads often depend on only a few recurring dimension attributes, such as grouping columns or highly selective filter attributes. Instead of denormalizing entire dimensions, we selectively graft only these useful attributes onto the foreign-key side of the fact table. This is the central idea behind ColumnCast: selectively materialize the few dimension attributes that repeatedly matter, under a storage budget, so the system can preserve a normalized logical schema while recovering much of the runtime benefit of denormalization. Depending on query shape, a grafted attribute can either eliminate a dimension join altogether or allow a predicate to be pushed down directly into the fact-table scan.

This design differs from execution-layer approaches such as sideways information passing and semijoin-based filtering, which reduce join costs by constructing and propagating filters at runtime []. Recent systems such as Parachute [8] and RPT+ [] further advance this direction. These methods remain valuable, but their coordination costs recur on every query. Our setting instead targets predictable analytical workloads where it is preferable to pay bounded storage overhead once in exchange for repeated runtime savings.

The key challenge is selectivity: not every graft is worthwhile, and using every available graft would recreate the drawbacks of OBTs. ColumnCast therefore operates in two stages. First, an offline optimizer analyzes workload logs and selects a budget-feasible set of

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

$Q_{4.1}$	$Q_{4.1}'$ (2 joins eliminated, 1 pred. push down)
SELECT c_nation, s_nation, d_year, sum(lo_revenue) FROM lineorder, date, customer, supplier WHERE lo_custkey = c_custkey AND lo_suppkey = s_suppkey AND lo_orderdate = d_datekey AND c_region = 'ASIA' AND s_region = 'ASIA' AND d_year >= 1992 AND d_year <= 1997 GROUP BY c_nation, s_nation, d_year	SELECT lo_d_year AS d_year, c_nation, SUM(lo_revenue - lo_supplycost) AS profit FROM customer, part, lineorder WHERE lo_custkey = c_custkey AND lo_partkey = p_partkey AND lo_c_region = 'AMERICA' AND lo_s_region = 'AMERICA' AND (p_mfgr = 'MFGR#1' OR p_mfgr = 'MFGR#2') GROUP BY lo_d_year, c_nation ORDER BY lo_d_year, c_nation;

Figure 2: Grafted dimensions in SSB Query 4.1 (57% speedup).

grafts that maximizes expected utility. Second, an online graft-aware rewriter uses those grafts only when doing so improves execution. Thus, the system is selective twice over: selective in what it materializes, and selective in when it uses the materialized attributes.

We demonstrate that with these strategies, on the Star Schema Benchmark(SSB) [6], under a modest storage budget, the best grafts are a small set of low-cardinality attributes related to date and geography. Across the workload, these selective grafts recover much of the performance advantage of denormalization without incurring its full storage cost. Fig. 1 corroborates this against: the vanilla dataset with no columns grafted, and SSB++, which grafts all possible columns that are referenced in any query. ColumnCast achieves an overall 1.41× speedup with about 8% storage overhead, which is 76% of SSB++'s performance with 5.5 × less storage overhead. It also avoids any query regression as in SSB+.

In summary, in this paper, we make the following contributions:

- introduce ColumnCast, a storage-layer framework for selective, attribute-level grafting;
- formulatethe problem of graft selection as a storage-constrained workload optimization
- evaluate ColumnCast against A and B ...

We conclude with future work in Sec. 3.

2 ColumnCast

We begin with an example to formalize the optimization problem (Sub. 2.1), then present our two-phase grafting approach (Sub. 2.2) and preliminary evaluation (Sub. refsec: evaluation) on DuckDB[5].

2.1 Example & Problem Formalization

Consider SSB Q 4.1 (Fig. 2). The query joins the fact table `lineorder` with the `date`, `customer`, `supplier`, and `part` dimensions, groups by `d_year` and `c_nation`, and filters on `c_region`, `s_region`, and `p_mfgr`. Although the schema is normalized, the query touches only a small number of dimension attributes, several of which are low-cardinality and highly compressible.

Observation 1. If all referenced attributes of a dimension are available in the fact table, that join can be eliminated. For example, if a query only needs `d_year` from date, then grafting `d_year` into `lineorder` removes the date join.

Observation 2. When a join cannot be eliminated, grafting a predicate attribute is still useful. In Q4.1, grafting `c_region` and `s_region` allows these predicates to be evaluated directly during the fact-table scan. This enables efficient zone-map pruning in engines such as DuckDB and reduces the data entering the remaining joins. Fig. 2 shows the rewritten query. With just these three grafts (`d_year`, `c_region`, and `s_region`), the database grows by only 5%, while the query speeds up by 57%.

These two observations lead directly to the optimization problem solved by ColumnCast. Let $R_i(q)$ be the set of attributes of dimension D_i referenced by query q, and let $P_i(q) \subseteq R_i(q)$ be the subset used in predicates. For an equally weighted workload W, we define the utility of a G of grafts as $U(G) = \sum_{q \in W} \Delta(q, G)$, wehre

$$\Delta(q, G) = \sum_i \mathbf{1}[R_i(q) \subseteq G] + \sum_i \big(1 - \mathbf{1}[R_i(q) \subseteq G]\big) \cdot |P_i(q) \cap G|,$$

The first term counts join eliminations and the second counts predicate pushdowns for dimensions whose joins remain.

Each graft also increases storage. We model this with a normalized weight for an attribute definfed as: $w(a) = \frac{|F|}{S(F)} \frac{\lceil \log_2 |(a)| \rceil}{8} + \mu_{(a)}$.

Therefore, we have the following optimization problem:

$$G^* = \arg\max_{G \subseteq C} U(G) \quad \text{s.t.} \sum_{a \in G} w(a) \leq \beta. \tag{1}$$

Here, $|F|$ is the fact-table cardinality, $S(F)$ is the base fact-table size, $\mu_{(a)}$ captures residual storage-engine overheads such as metadata and compressional overheads, and β is the normalized storage budget.

For SSB at SF300, $|F| = 1.8 \times 10^9$. Using DuckDB storage statistics with block size 262,144 bytes and 242,262 used blocks, the fact-table footprint is $S(F) = 242{,}262 \times 262{,}144 \approx 63.51 \times 10^9$ bytes ($\approx$ 41.85 GiB). This gives $\frac{|F|}{S(F)} \approx \frac{1.8 \times 10^9}{41.85 \times 10^9} \approx 0.0430$. We distinguish three classes of attributes based on their cardinality and metadata: Class 1 for `p_mfgr`, `d_year`, `s_region`, and `c_region`, with $\mu_1 = 0.05$; Class 2 for `d_yearmonth`, `s_nation`, `c_nation`, and `p_category`, with $\mu_2 = 0.10$; and Class 3 for `s_city`, `c_city`, and `p_brand1`, with $\mu_3 = 0.20$. We show the estimated weights for the SSB graft candidates in [**?**].

2.2 Implementation: Two-Phase Grafting

ColumnCast has two phases: an offline *Column Grafter* that selects a budget-feasible set of grafts, and an online graft-aware rewriter that substitutes grafted attributes when beneficial. DuckDB still chooses the final physical plan.

Offline phase. The Column Grafter enumerates candidate attributes over PK–FK joins, estimates their storage cost and workload utility, and selects a set under a budget. The subsequent materialization uses standard `ALTER TABLE`. In our current prototype, any non-key attribute on the primary-key side is a valid candidate. For SSB, `lineorder` joins with date, customer, supplier, and part, so attributes from these dimensions constitute the candidate pool.

Storage is estimated using the normalized weight $w(a)$ from the previous subsection. A graft set G is feasible if $\sum_{a \in G} w(a) \leq \beta$.

Utility is estimated from the same two effects used in our formalization: join elimination and predicate pushdown. A candidate is useful if it helps fully cover a dimension and remove a join, or if it enables selective predicate pushdown when the join remains. Low-cardinality filter attributes are therefore strong candidates, while high-cardinality projection-only attributes are usually poor ones.

Online phase. At query time, if the grafted attributes fully cover a dimension, the join is eliminated. If not, the rewriter uses grafted

Query	d_year	s_region	c_region	s_nation	Overall
1.1	35.70%	–	–	–	**35.70%**
1.2	–	–	–	–	2.94%
1.3	–	–	–	–	0.36%
2.1	11.05%	58.98%	–	–	**76.55%**
2.2	12.81%	33.14%	–	–	**50.19%**
2.3	13.01%	28.26%	–	–	**44.94%**
3.1	18.26%	11.27%	2.33%	10.93%	**49.36%**
3.2	2.07%	–	–	36.59%	**39.42%**
3.3	10.70%	–	–	–	**10.70%**
3.4	–	–	–	–	1.72%
4.1	7.76%	18.01%	21.23%	–	**57.38%**
4.2	–	14.25%	42.87%	–	**63.23%**
4.3	–	–	23.63%	–	**23.63%**

Table 1: Speedup attribution by grafted attribute.

columns only for highly selective point or range predicates, and only for simple or conjunctive predicates. Otherwise, the query is left unchanged and DuckDB optimizes it as usual.

Thus, the offline phase decides *what* to graft, and the online phase decides *when* to use it.

2.3 Preliminary Evaluation

We evaluate how COLUMNCAST incrementally grafts promising attributes into `lineorder`. All experiments were run in DuckDB (v1.4.4) on a server with 4× Intel Xeon E5-4640 v2 CPUs (40 physical cores, 80 hardware threads) at 2.20 GHz, 125 GiB RAM, and a 7.68 TB Samsung PM893 SATA SSD. DuckDB already supports min-max filter transfer on join keys from the hash-table build side to the probe side [**?**], as well as multiple compression techniques [7]. We ran each SSB query with 128 threads and a 96 GiB memory limit. We report the median of runs 2–11 (11 total) to exclude warm-up.

We add grafts to `lineorder` until storage overhead approaches 10%. After each graft, we rerun the full benchmark and decide per query whether to use the grafted or original column. We treat changes below 5% as noise.

Our offline analysis identifies `d_year` from `date` as the strongest first candidate. Grafting it removes joins in 8 queries and yields a maximum speedup of 26% on Q1.1 at only 1.82% storage overhead. The next candidate is `s_region`, which removes joins in 6 queries and yields a maximum speedup of 43% on Q2.1 with an additional 1.69% storage overhead. We then graft `c_region` and `s_nation`. After all four grafts, 10 of the 13 queries improve, with speedups ranging from 10% to 77%. For the remaining queries, grafting causes no meaningful slowdown. This suggests that grafting does not hurt unrelated queries. Fig. 3 summarizes the speed–storage tradeoff.

Ablation Study. Table 1 breaks down per-query speedups by incremental updates to the set of graphs. The largest gains come from grafts that eliminate selective dimension joins on predicates. `s_region` drives the Q2 family, contributing 58.98% on Q2.1, 33.14% on Q2.2, and 28.26% on Q2.3. `d_year` is the main driver for Q1.1, contributing 35.70%. For more complex queries, gains come from multiple grafts. For example, Q3.1 benefits from all four attributes, with `d_year` and `s_nation` contributing the most. Q4.1 and Q4.2 are driven mainly by `c_region` (21.23% and 42.87%, respectively), showing the benefit of pushing region filters into the fact table and reducing join work.

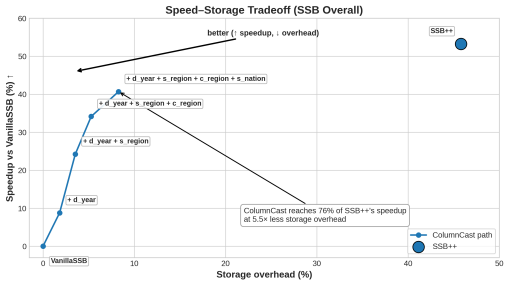

Figure 3: Speed–storage tradeoff on SSB (SF=300).

TPC-H. We evaluate COLUMNCAST on TPC-H Q9 (the slowest TPC-H query in DuckDB [3]). We graft `r_name` into `supplier`, `n_name` into `customer`, and `c_custkey` and `o_orderdate` into `lineitem`. This yields a 50% speedup at 10% storage overhead, consistent with the SSB results. A broader TPC-H evaluation remains future work.

3 Conclusion and Future Work

We introduced COLUMNCAST, a lightweight approach for reducing join cost through selective grafting. Our early results are encouraging and point to a broader systems question: how should a database expose and optimize selective denormalization as a design choice?

A natural next step is deeper optimizer integration. In this paper, graft selection and query rewriting are external to the optimizer. A graft-aware cost-based optimizer could reason about equivalences between base and grafted attributes, eliminate redundant PK–FK joins, choose between fact-side and dimension-side predicate evaluation, and enable further rewrites such as aggregation pushdown. But the deeper issue is that grafting changes more than access paths. It changes the logical opportunities available to the optimizer and the physical tradeoffs beneath them. This suggests that the right abstraction is not a local rewrite rule, but a joint optimization problem over query plans and storage layout. Our current prototype grafts attributes greedily under a storage budget. A fuller formulation would jointly optimize which attributes to graft, where to store them, how to compress them, and when to retire them as the workload changes. Our preliminary investigation suggests that query trees offer a natural way forward, since grafting changes both plan structure and operator cost, and can unlock rewrites whose value depends on the surrounding plan context.

This, in turn, makes grafting a lifecycle problem rather than a one-time design decision. Grafts must remain correct and useful under inserts, deletes, and updates to both fact and dimension tables. This raises questions of incremental maintenance cost, consistency guarantees, and when stale grafts should be recomputed or invalidated. At the same time, the value of a graft is workload-dependent. Our current setting assumes a stable workload, but real workloads evolve. Previously useful grafts may become dead weight, while new hot predicates may justify new grafts. An important open problem is therefore how to detect workload drift and adapt the graft set without oscillation or excessive reorganization. In this sense, grafting belongs naturally within dynamic physical design rather than static schema tuning.

Another broader question is how far this idea extends across schemas and systems. Our current setting focuses on one-to-many PK–FK joins, where the semantics are clean and the benefit of join elimination is easy to isolate. Many real workloads, however, contain bridge tables, and many-to-many joins. In such settings, grafting may still reduce join work, but it also risks larger storage blowup and more complex maintenance. The key question is no longer only whether a graft can remove a join, but when the extra denormalization remains worthwhile before maintenance costs dominate.

The same issue appears at the systems level. COLUMNCAST is complementary to SIP-style techniques such as Plaque[4], Parachute[8], and RPT+. More broadly, it would be useful to understand how grafting interacts with zonemaps, late materialization, join ordering, partitioning, and column compression, and whether these effects are additive or overlapping. These interactions may be even more important in distributed engines. There, dimension joins often trigger shuffles, repartitioning, and coordination, so eliminating them may save not only local CPU work but also network and synchronization cost. At the same time, grafts may increase replication and maintenance overhead. Studying this tradeoff in systems such as AsterixDB[1] and SparkSQL[2] is an important next step.

Finally, COLUMNCAST suggests a broader role in self-driving database systems. Rather than treating grafting as a one-off tuning step, a self-driving system could continuously decide when to create, retain, or remove grafts under explicit storage and maintenance budgets. This raises broader questions about observability, stability, and control for workload-adaptive storage design, and suggests that grafting may be useful not only as a performance optimization, but also as a new actuator for autonomous physical design.

References

[1] S. Alsubaiee, Y. Altowim, H. Altwaijry, A. Behm, V. R. Borkar, Y. Bu, M. J. Carey, I. Cetindil, M. Cheelangi, K. Faraaz, E. Gabrielova, R. Grover, Z. Heilbron, Y. Kim, C. Li, G. Li, J. M. Ok, N. Onose, P. Pirzadeh, V. J. Tsotras, R. Vernica, J. Wen, and T. Westmann. AsterixDB: A scalable, open source BDMS. *Proc. VLDB Endow.*, 7(14):1905–1916, 2014.

[2] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia. Spark sql: Relational data processing in spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, SIGMOD '15, page 1383–1394, New York, NY, USA, 2015. Association for Computing Machinery.

[3] M. Dreseler, M. Boissier, T. Rabl, and M. Uflacker. Quantifying TPC-H choke points and their optimizations. *Proc. VLDB Endow.*, 13(8):1206–1220, 2020.

[4] Y. Lin and S. Mehrotra. Plaque: Automated predicate learning at query time. *Proc. ACM Manag. Data*, 2(1), Mar. 2024.

[5] H. Mühleisen and M. Raasveldt. DuckDB. https://duckdb.org, 2026. Version 1.4.4.

[6] P. E. O'Neil, E. J. O'Neil, X. Chen, and S. Revilak. The star schema benchmark and augmented fact table indexing. In *Performance Evaluation and Benchmarking: First TPC Technology Conference (TPCTC), Revised Selected Papers*, pages 237–252. Springer, 2009.

[7] M. Raasveldt. Lightweight compression in duckdb, Oct. 2022. DuckDB Blog.

[8] M. Stoian, A. Zimmerer, S. Krid, A. L. Ngom, J. Ding, T. Kraska, and A. Kipf. Parachute: Single-pass bi-directional information passing. *Proc. VLDB Endow.*, 18(10):3299–3311, June 2025.